ALUMNICONNECT

BUSINESS RULES CATALOGUE

Requests | Allocations | Availability | Booking | Sessions | Reviews

DBMS Mini Project - Alumni-Student Mentorship and Mock Interview Platform

Implementation-oriented specification for MySQL 8.x / InnoDB

Version 1.0 | 02 November 2026

Prepared from the supplied Alumni-Student Mentorship Platform: DBMS Project Blueprint
Expanded with implementation guidance aligned to DBMS Units 1-4

Document Control and Reading Guide

Item	Specification
Document purpose	Define the business rules and database enforcement mechanisms for AlumniConnect.
Primary audience	Project team, database designer, backend developers, testers and project evaluators.
System boundary	Student/Mentee portal, Alumni/Mentor portal, Administrator portal, one modular backend and specialised data stores.
Primary source of truth	MySQL 8.x with InnoDB. Redis and Neo4j are supporting stores, not authorities for booking or identity decisions.
Status	Design specification. SQL examples are implementation templates and must be aligned with the final physical schema.
Version	1.0 - 02 November 2026

How to interpret the rules

Label	Meaning
[B] Blueprint-aligned	Explicitly stated in the supplied AlumniConnect blueprint.
[R] Recommended rule	A concrete policy proposed to make the requirements implementable; confirm configurable choices with the team.
[T] Technical enforcement	The database or application mechanism recommended to enforce the rule.

Important distinction

The blueprint establishes the product direction and several key invariants. It does not specify every policy value (for example, cancellation cutoff, maximum mentor load or review-edit window). Such values are marked as configurable decisions rather than presented as already-approved requirements.

Contents

Sections 1-9	Sections 10-18
1. Executive Summary and Design Basis	2. Domain Model and Core Definitions
3. Global Identity, Access and Profile Rules	4. Mentorship Request Rules and Lifecycle
5. Allocation Rules	6. Availability and Booking Rules
7. Session, Feedback and Review Rules	8. Notifications, Audit and Administration Rules
9. Rule Enforcement Matrix and Relational Constraints	10. Trigger Design and MySQL Trigger Examples
11. Stored Procedure Catalogue and Implementation Examples	12. Transactions, ACID, Isolation and Concurrency
13. Views, Functions, Indexes and Advanced SQL	14. Redis, Neo4j and Cross-Database Consistency
15. Security, Privacy and Data Lifecycle	16. Verification and Acceptance Test Catalogue
17. Policy Decisions to Finalise	18. Implementation Checklist and Source Basis

1. Executive Summary and Design Basis

AlumniConnect is a mentorship, career-guidance and mock-interview platform. Students discover relevant alumni, submit mentorship requests, schedule sessions, attend offline meetings, receive feedback and track progress. Alumni manage their professional profiles and availability, respond to requests, and record session outcomes. Administrators manage users, departments, skills, mentorship domains and platform analytics.

The central design recommendation is a modular monolith with polyglot persistence: one backend application with clearly separated business modules, MySQL as the authoritative relational database, Redis as a performance layer and Neo4j as a recommendation graph. This avoids splitting tightly related booking and allocation workflows across independently deployed services.

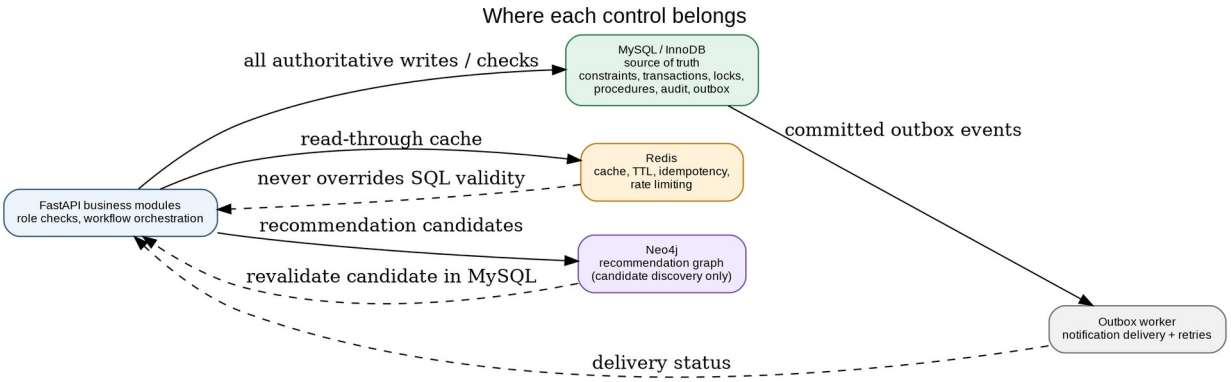


Figure 1. Each technology has a bounded responsibility; SQL remains authoritative for business decisions.

1.1 What this catalogue is intended to protect

- Correct lifecycle transitions: pending requests cannot be accepted twice; cancelled requests cannot later create allocations.
- Referential consistency: an allocation must correspond to the request that created it; a session must correspond to a valid booking.
- Scheduling correctness: one slot cannot be actively booked twice, and neither participant can have overlapping confirmed appointments.
- Trustworthy feedback: only actual participants can submit feedback, and mentor reviews require a completed session.
- Reliable side effects: notifications are queued within the same SQL transaction as the business change and delivered after commit.
- Concurrency safety: concurrent requests and bookings must not bypass validation due to timing or race conditions.

1.2 Normative terms

MUST means a non-negotiable integrity or security requirement. SHOULD means the recommended default, with deviations documented. MAY means an optional enhancement. A rule labelled [R] is a proposed decision and should be confirmed by the team before implementation.

1.3 Source-aligned product scope

Actor	Core responsibilities
Student / Mentee	Maintain student profile; discover mentors; send and track requests; book mentorship or mock-interview sessions; view outcomes; submit feedback and mentor reviews.
Alumni / Mentor	Maintain professional profile, skills and experience; publish availability; accept or reject requests; manage bookings; record session status and outcomes.
Administrator	Manage accounts, roles, departments, skills and domains;

Actor	Core responsibilities
	moderate platform data; monitor requests, allocations, sessions, reviews and analytics.

2. Domain Model and Core Definitions

The following entities form the business vocabulary used throughout the catalogue. They represent logical concepts from the blueprint; exact table names and columns may be adjusted to the team's final schema.

Concept	Meaning	Why it is separate
User Account	One authentication identity with a role and account status.	Avoids duplicating email, credentials and account lifecycle across Student and Mentor records.
Student Profile	Student-specific academic details and preferences linked to one account.	Separates student attributes from common identity fields.
Mentor Profile	Alumni professional details, company, experience, availability and mentoring capacity.	Separates professional information from common identity fields.
Mentorship Request	A student's request to a specific mentor for a domain or objective.	Tracks intent and decisions before an actual mentorship relationship exists.
Mentor Allocation	The accepted student-mentor relationship created from a request.	Represents an active or ended mentoring relationship; not merely a recommendation.
Recommendation Run / Candidate	A stored recommendation calculation and its suggested mentors.	Allows the system to explain and compare recommendations without treating them as accepted allocations.
Mentor Availability	A time window published by a mentor.	Represents open scheduling capacity before a student reserves it.
Booking	A planned appointment against an availability slot and valid allocation.	Represents a reservation, not proof that a meeting happened.
Mentorship Session	The actual planned/occurred meeting record connected to a booking.	Captures completion, no-show/cancellation, topics and outcomes separately from the reservation.
Session Feedback	Structured feedback from an actual participant about a session.	Can support feedback from either participant.
Mentor Review	A student's rating and review of a mentor following a completed session.	A distinct rating object used for mentor reputation and analytics.
Notification Outbox	A durable event waiting for asynchronous delivery.	Prevents notifications from being emitted for rolled-back database operations.
Audit Log	Append-only evidence of important data and state changes.	Supports traceability, debugging and administrator accountability.

2.1 Critical distinctions

Request is not allocation

A recommendation is only a candidate. A student request is a formal invitation. An allocation exists only after the mentor accepts the pending request and the database commits the request status update and allocation insert together.

Booking is not session

A booking is a planned appointment. A session records what actually happened: completed, cancelled or no-show, together with topics assessed, observations, improvement areas and next steps.

Feedback is not the same as a mentor review

Session feedback may be submitted by either participant. A mentor review is the student's rating of the mentor after a completed session. Store and report these separately.

3. Global Identity, Access and Profile Rules

These rules apply to every module. Authorization must be enforced in the backend and, where applicable, reinforced through database privileges and constraints.

3.1 Account and identity rules

Rule ID	Business rule	Primary enforcement / evidence
ACC-001 [R]	Each account MUST have a unique, normalized email or institutional login identifier. Case and whitespace normalization must be consistent.	UNIQUE normalized identifier; server-side normalization; duplicate-registration test.
ACC-002 [B/R]	One common User Account owns authentication. Student Profile and Mentor Profile are role-specific extensions linked to that identity.	PK/FK one-to-zero-or-one relationship; unique user_id in each profile table.
ACC-003 [T]	The acting user ID and role MUST be derived from the authenticated server-side session/token, not trusted from a request body.	FastAPI authorization dependency; negative API tests.
ACC-004 [R]	Only ACTIVE accounts may submit requests, accept requests, publish slots, book, record outcomes or review.	Account-status guard in each write procedure/service.
ACC-005 [B/R]	A user may access only role-appropriate functions. An administrator has administrative permissions but does not silently become a student or mentor.	Role/permission checks; deny-by-default tests.
ACC-006 [R]	Account suspension blocks new activity but MUST NOT erase existing requests, allocations, bookings, sessions or reviews.	Soft status; restrictive FKs; suspension workflow.
ACC-007 [R]	Administrative role changes and account suspension/reactivation MUST be recorded with actor, timestamp and reason.	Admin procedure + audit log.

3.2 Profile, skill and domain rules

Rule ID	Business rule	Primary enforcement / evidence
PRO-001 [B/R]	A student can associate with multiple skills and mentorship-domain interests; a mentor can associate with multiple skills and areas of expertise.	Junction tables with composite UNIQUE(student_id, skill_id) and UNIQUE(mentor_id, skill_id).
PRO-002 [R]	A mentor is eligible for discovery only when the account is active and the profile is visible/approved and sufficiently complete.	MySQL eligibility query/view; profile status.
PRO-003 [B]	Departments, companies, skills and mentorship domains are managed as	FKs to taxonomy tables; administrator-only write permission.

Rule ID	Business rule	Primary enforcement / evidence
	controlled reference data.	
PRO-004 [R]	A mentor cannot publish availability when inactive, suspended or not permitted to mentor.	Availability procedure checks mentor account/profile status.
PRO-005 [R]	Changing skills, domains, experience or visibility MUST invalidate affected mentor-cache and recommendation-cache entries after the SQL update commits.	Outbox/cache invalidation; Redis TTL as fallback.
PRO-006 [R]	Public discovery results MUST exclude private contact data and fields not intended for students.	Least-privilege view/API projection; privacy tests.

4. Mentorship Request Rules and Lifecycle

A student discovers a mentor and creates a request containing the intended domain, objective or help required. The request starts in PENDING and is acted on by the addressed mentor. The blueprint's explicit terminal decision paths are Accepted, Rejected and Cancelled.

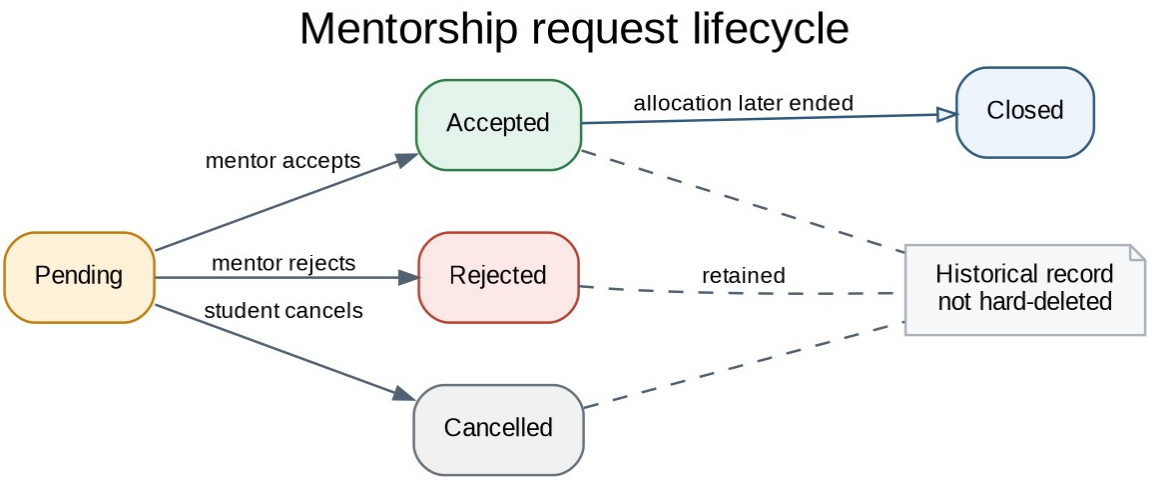


Figure 2. A request has one decision path. Rejected and cancelled requests remain in history.

4.1 Request rule catalogue

Rule ID	Business rule	Primary enforcement / evidence
REQ-001 [B]	A request MUST identify one student, one target mentor and the relevant mentorship domain/objective.	NOT NULL columns; FKs; input validation.
REQ-002 [R]	A new request MUST be created with status PENDING and a server-generated timestamp.	DEFAULT status; procedure controls insert.
REQ-003 [B]	Only the addressed mentor may accept or reject the request.	Actor-to-mentor ownership check in API/procedure.
REQ-004 [B]	The student who created a pending request may cancel it.	Actor-to-student ownership check.
REQ-005 [B]	Permitted transitions are PENDING -> ACCEPTED, PENDING -> REJECTED and PENDING -> CANCELLED.	Transition validation in procedure; optional status trigger as last-line guard.
REQ-006 [R]	ACCEPTED, REJECTED and CANCELLED are terminal request states. A later change	Reject invalid transition; audit old/new state.

Rule ID	Business rule	Primary enforcement / evidence
	must use a new request or a separate allocation lifecycle action.	
REQ-007 [R]	A student MUST NOT create duplicate pending requests for the same mentor and domain unless the previous request is resolved or explicitly reopened by policy.	Procedure-level check under student/mentor lock; optional generated active-request key.
REQ-008 [B]	An accepted request may create at most one allocation.	UNIQUE(mentor_allocation.request_id).
REQ-009 [B]	Rejected and cancelled requests MUST remain available for history, reporting and funnel analytics.	No hard delete; append-only state history/audit.
REQ-010 [B]	Acceptance, allocation creation, request status update and notification-outbox insertion MUST commit or roll back as one SQL transaction.	Stored procedure; InnoDB transaction; failure-injection test.
REQ-011 [R]	A request must not be accepted if the mentor account/profile is inactive or its mentoring capacity is already full.	Lock mentor profile; verify capacity before insert.
REQ-012 [R]	A request decision must record decision time and, for rejection, an optional reason code/message.	decision_at, decision_reason columns; audit.

4.2 Request transition table

Current state	Action / actor	Next state	Allowed?	Notes
No request	Student submits	PENDING	Yes	Create one new request.
PENDING	Mentor accepts	ACCEPTED	Yes	Creates exactly one allocation atomically.
PENDING	Mentor rejects	REJECTED	Yes	Retain record; notify student.
PENDING	Requesting student cancels	CANCELLED	Yes	Retain record; no allocation.
ACCEPTED	Student cancels request	CANCELLED	No	End the allocation through its own lifecycle.
REJECTED / CANCELLED	Any actor changes to ACCEPTED	No	No	Terminal state; create a new request if appropriate.
ACCEPTED	Mentor accepts again	No	No	Idempotent response should report already accepted; never create a second allocation.

4.3 Required acceptance behaviour

1. Lock the relevant student and mentor records in a consistent order, then lock and re-read the request row.
2. Verify that the request is still PENDING and the authenticated actor is the target mentor.
3. Check that mentor capacity and any student-side allocation policy are satisfied.
4. Insert the allocation with the request ID protected by a unique constraint.
5. Update the request to ACCEPTED and set decision metadata.
6. Insert the student notification into the outbox before COMMIT. A worker sends it only after commit.

5. Allocation Rules

An allocation represents the accepted mentoring relationship. It is not interchangeable with a request, a recommendation result or an individual session.

Rule ID	Business rule	Primary enforcement / evidence
ALL-001 [B]	An allocation MUST be created only from a request whose status is PENDING at the moment it is accepted.	Acceptance procedure + row lock.
ALL-002 [B]	One request can produce no more than one allocation.	UNIQUE(request_id).
ALL-003 [R]	Student and mentor IDs on the allocation MUST exactly match the originating request.	Populate from the locked request row; do not accept party IDs from client input.
ALL-004 [B/R]	An allocation begins in ACTIVE state and may later be ENDED.	CHECK status; controlled close procedure.
ALL-005 [B/R]	A mentor's active allocation count MUST NOT exceed configured mentoring capacity.	Lock mentor_profile row and count ACTIVE allocations before acceptance.
ALL-006 [R]	The number of simultaneous active allocations allowed for one student is a configurable policy, optionally scoped by domain.	Config value + student-row lock; do not hard-code until approved.
ALL-007 [R]	Ending an allocation MUST record ended_at, ended_by and a reason.	Close procedure; NOT NULL validation when status=ENDED.
ALL-008 [R]	An ended allocation cannot create new bookings. Existing historical bookings and sessions remain readable.	Booking procedure checks allocation.status=ACTIVE.
ALL-009 [R]	Allocation records MUST NOT be physically deleted as a normal business operation.	Soft lifecycle + RESTRICT FKs + audit.

5.1 Allocation lifecycle

State	Permitted operation	Result
ACTIVE	Mentor or authorised administrator closes the relationship under policy.	Move to ENDED; record timestamp, actor and reason.
ENDED	Attempt new booking	Reject. Historical sessions and reviews remain available.
ACTIVE	Accept same request again	Reject or return already-processed result; no duplicate allocation.

6. Availability and Booking Rules

Scheduling is the highest-risk consistency area because multiple students may attempt to reserve the same mentor slot and a student may attempt to book two different mentors for overlapping times. The blueprint explicitly requires the system to check slot availability, student overlap, mentor capacity, valid allocation/request and duplicate submission before confirming a booking.

6.1 Availability rules

Rule ID	Business rule	Primary enforcement / evidence
AVL-001 [B]	A mentor publishes their own availability slots.	Actor ownership check; FK mentor_id.
AVL-002 [R]	Every slot MUST have start_at < end_at and an explicit time-zone interpretation. Store instants consistently (recommended UTC) and display in the user's local zone.	CHECK(end_at > start_at); API timezone validation.
AVL-003 [R]	A mentor MUST NOT publish overlapping active slots unless the product explicitly supports parallel/group sessions.	Availability procedure checks overlap while mentor profile row is locked.
AVL-004 [R]	A slot begins OPEN; after a confirmed reservation it becomes BOOKED. A mentor may block an unbooked slot.	Status CHECK + procedures.
AVL-005 [R]	A booked slot's time boundaries cannot be edited directly. Use a controlled cancellation/reschedule workflow.	BEFORE UPDATE trigger or procedure-only grants; audit.
AVL-006 [R]	A slot whose start time has passed cannot be newly booked.	Booking procedure checks start_at > CURRENT_TIMESTAMP.
AVL-007 [R]	A slot should not be deleted if it has historical bookings.	ON DELETE RESTRICT; mark BLOCKED/EXPIRED instead.

6.2 Booking rules

Rule ID	Business rule	Primary enforcement / evidence
BOOK-001 [B/R]	A booking MUST belong to an active allocation (recommended core policy) and the allocation's mentor must own the selected slot.	Join allocation to slot; procedure checks both IDs and ACTIVE status.
BOOK-002 [B]	The selected slot must still be OPEN at the time of commit.	SELECT ... FOR UPDATE on mentor_availability row.
BOOK-003 [B]	The same slot cannot have more than one active booking.	Generated active_slot_id + UNIQUE index; slot row lock.
BOOK-004 [B]	The student must not have another overlapping confirmed booking.	Conflict query + student_profile row lock.
BOOK-005 [B]	The mentor must not have another overlapping confirmed booking.	Conflict query + mentor_profile row lock.
BOOK-006 [B]	Mentor capacity must not be exceeded. For a one-to-one slot, slot capacity is one.	Slot-level unique active booking; mentor capacity check.
BOOK-007 [B]	A booking submission must not be processed twice because of retries, double-clicks or network timeouts.	Idempotency key with UNIQUE(actor_id, idempotency_key); return prior result.
BOOK-008 [B]	Booking creation and slot reservation must be atomic.	One InnoDB transaction.
BOOK-009 [R]	Only the student who owns the allocation may book on the student side.	Resolve student profile from authenticated actor.
BOOK-010 [R]	A booking becomes CONFIRMED only as part of a successful committed transaction.	Procedure returns success after COMMIT.
BOOK-011 [R]	Cancellation is a status change; the booking row is retained for history.	UPDATE status; no DELETE.
BOOK-012 [R]	A cancelled future booking releases the slot only when the cancellation policy	Cancellation procedure updates booking and slot in one transaction.

Rule ID	Business rule	Primary enforcement / evidence
	permits re-opening it.	
BOOK-013 [R]	Rescheduling must preserve the original booking history and link the replacement to the original.	New booking + rescheduled_from_booking_id; do not overwrite timestamps silently.
BOOK-014 [R]	A confirmed booking cannot be directly edited to a different participant or slot.	Procedure-only update privileges; cancel/reschedule workflow.
BOOK-015 [R]	All displayed times must be derived from stored start/end instants and a declared time zone, not from client-provided text.	Backend time conversion and validation.

6.3 Time-overlap definition

Use half-open intervals

Treat appointments as [start_at, end_at). Two intervals overlap exactly when existing.start_at < requested.end_at AND existing.end_at > requested.start_at. This allows a 10:00-10:30 session to be followed by a 10:30-11:00 session without a false conflict.

Overlap predicate used inside the booking transaction

```
SELECT COUNT(*) INTO v_conflicts
FROM booking b
WHERE b.status = 'CONFIRMED'
  AND b.start_at < v_requested_end
  AND b.end_at   > v_requested_start
  AND (b.student_id = v_student_id
       OR b.mentor_id = v_mentor_id);

IF v_conflicts > 0 THEN
  SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Student or mentor has an overlapping booking';
END IF;
```

6.4 Atomic booking sequence

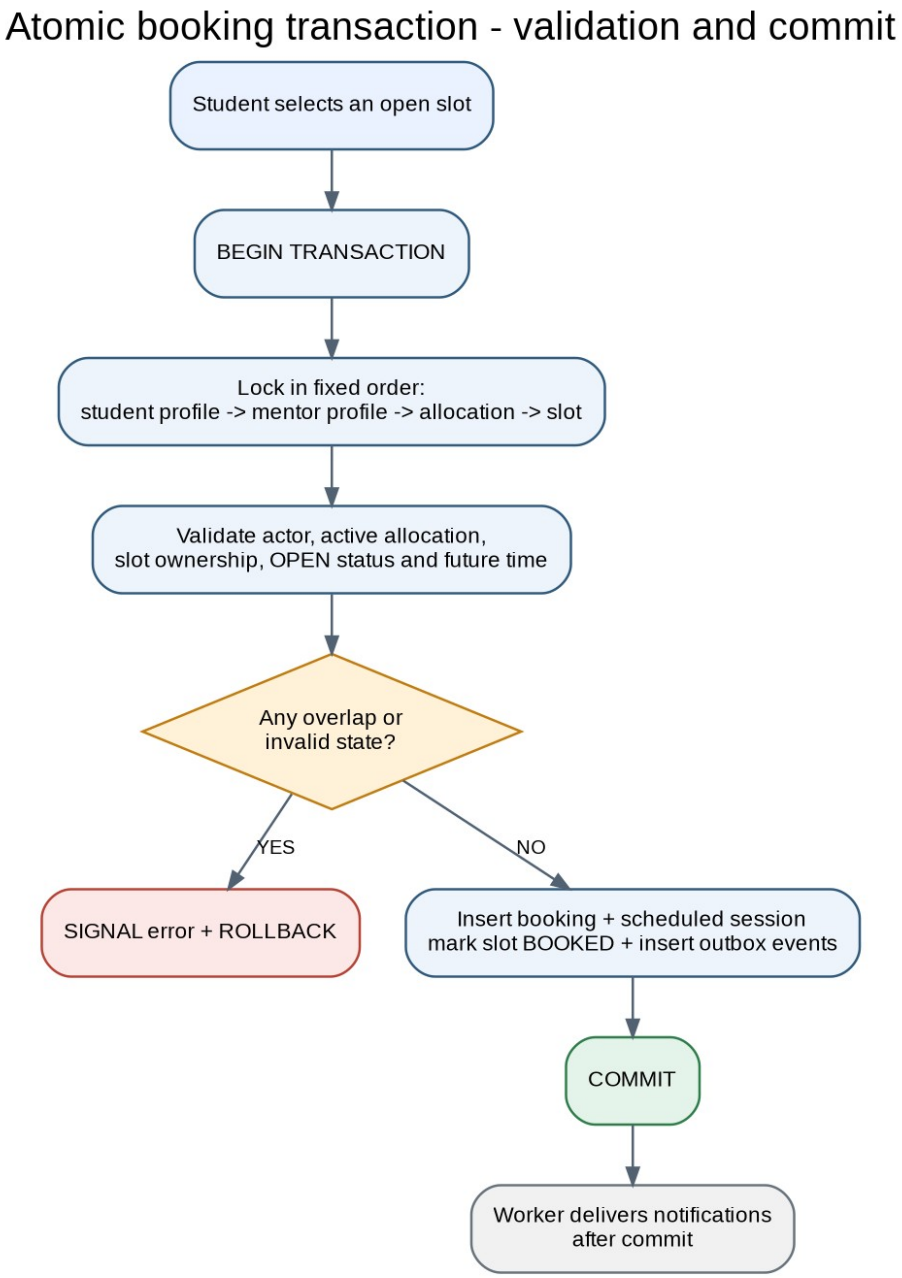


Figure 3. A booking is confirmed only after all validation and writes commit together.

7. Session, Feedback and Review Rules

7.1 Session rules

Rule ID	Business rule	Primary enforcement / evidence
SES-001 [B]	A session is a distinct record from its booking. The booking represents the planned appointment; the session represents the meeting and outcome.	Separate booking and mentorship_session tables.
SES-002 [B/R]	A confirmed booking creates at most one session record, initially SCHEDULED.	UNIQUE(session.booking_id); insert in booking transaction.
SES-003 [B]	Allowed session outcomes are SCHEDULED -> COMPLETED, CANCELLED or NO_SHOW.	Controlled session-update procedure.

Rule ID	Business rule	Primary enforcement / evidence
SES-004 [B]	After the offline meeting, the mentor records whether it was completed and captures topics, skills assessed, observations, improvement areas, next steps and preparation.	Session outcome columns / child detail tables; required-field validation.
SES-005 [R]	Only the assigned mentor may record the session outcome; an administrator may correct a record through an audited override.	Actor-to-booking mentor ownership check.
SES-006 [R]	A terminal session state cannot be silently changed back to SCHEDULED.	State-transition procedure; audit log.
SES-007 [B/R]	Session history must be retained, including cancelled and no-show outcomes.	No hard delete; restrictive FKs.

7.2 Session feedback rules

Rule ID	Business rule	Primary enforcement / evidence
FB-001 [B]	Feedback cannot be submitted before the session has taken place.	Procedure checks session is COMPLETED.
FB-002 [B]	Only actual participants of the session may submit session feedback.	Resolve participants via session -> booking; compare authenticated actor.
FB-003 [R]	Each participant may submit at most one feedback record per session unless a formal revision model is implemented.	UNIQUE(session_id, submitted_by_user_id).
FB-004 [R]	Feedback content must comply with length and content validation; do not store secrets or unrelated sensitive data.	API validation, CHECK/LENGTH policy and moderation.
FB-005 [R]	Feedback history must not be overwritten without an audit trail.	Append-only or revision table; update restrictions.

7.3 Mentor review rules

Rule ID	Business rule	Primary enforcement / evidence
REV-001 [B]	A mentor review MUST reference a valid completed session.	FK + procedure / optional BEFORE INSERT trigger.
REV-002 [B]	Only the student who participated in the completed session may review the mentor.	Actor-to-student check against booking participant.
REV-003 [B]	A student may submit at most one mentor review per session.	UNIQUE(session_id, student_id).
REV-004 [R]	Rating must be an integer within the agreed range (recommended 1-5).	CHECK(rating BETWEEN 1 AND 5).
REV-005 [B/R]	A review must not be silently overwritten; corrections are versioned or handled through audited moderation.	No general UPDATE/DELETE permission; revision/audit table.
REV-006 [R]	Average rating must be derived from eligible reviews, not stored as the only authoritative value.	Aggregate view/function; optional cached summary with invalidation.
REV-007 [R]	Mentor cannot review themselves and student cannot submit review for another student's session.	Participant validation in procedure.

7.4 Why booking, session, feedback and review need separate records

Record	Question it answers	Example state
Booking	What appointment was reserved?	CONFIRMED / CANCELLED
Session	What happened at the appointment?	SCHEDULED / COMPLETED / CANCELLED / NO_SHOW
Session Feedback	What did a participant say about this session?	Submitted by student or mentor
Mentor Review	What rating did the student give this mentor after the completed session?	Rating 1-5 + review text

8. Notifications, Audit and Administration Rules

8.1 Notification rules

Rule ID	Business rule	Primary enforcement / evidence
NOT-001 [B]	A new mentorship request notifies the target mentor.	Outbox event REQUEST_SUBMITTED.
NOT-002 [B]	Request acceptance or rejection notifies the student.	Outbox event REQUEST_ACCEPTED / REQUEST_REJECTED.
NOT-003 [B]	Booking confirmation or cancellation notifies both participants.	Two recipient rows or one event with explicit recipient set.
NOT-004 [B]	Session reminders notify both participants.	Scheduled worker; reminder deduplication key.
NOT-005 [B]	Feedback submission notifies the relevant participant.	Outbox event FEEDBACK_SUBMITTED.
NOT-006 [B]	Outbox event insertion MUST be part of the business transaction; delivery occurs after commit.	MySQL outbox + background worker.
NOT-007 [R]	Delivery retries must be idempotent; duplicate email/push delivery should be suppressed by event ID.	UNIQUE(event_id, recipient); worker deduplication.
NOT-008 [R]	Notification delivery failure does not roll back a successful booking or acceptance.	Separate delivery state; retry and dead-letter handling.

8.2 Audit rules

Rule ID	Business rule	Primary enforcement / evidence
AUD-001 [B]	Important state changes must be recorded for request, allocation, booking, session, review moderation and account administration.	Audit log and/or immutable event log.
AUD-002 [R]	An audit record should include entity type, entity ID, action, actor, timestamp and relevant before/after values.	Audit schema; actor from authenticated business procedure.
AUD-003 [R]	Audit records are append-only for normal application roles.	Separate DB privileges; no UPDATE/DELETE grants.
AUD-004 [R]	Audit data must not include password hashes, tokens, secrets or unnecessary personal information.	Explicit allow-list for audit payload fields.

Rule ID	Business rule	Primary enforcement / evidence
AUD-005 [R]	Business events and technical logs are distinct: logs help debugging, audit records prove business changes.	Separate retention and access policy.

8.3 Administrator rules

Rule ID	Business rule	Primary enforcement / evidence
ADM-001 [B]	Administrator may manage accounts, departments, skills, mentorship domains and platform analytics.	Admin-only backend policy and DB grants.
ADM-002 [R]	Admin actions that change account status, role, taxonomy, session outcome or review visibility must record actor and reason.	Admin procedures + audit.
ADM-003 [R]	Administrators should not directly edit booking rows to bypass allocation, slot and overlap rules.	Admin uses controlled procedures; no direct DML grant.
ADM-004 [R]	Analytics views should aggregate data and avoid exposing private student/mentor contact details by default.	Read-only reporting views.
ADM-005 [R]	Administrative overrides must be explicit, permission-scoped and auditable; do not silently impersonate users.	Dedicated override reason and actor metadata.

9. Rule Enforcement Matrix and Relational Constraints

A business rule should be enforced at the lowest reliable layer that can express it. Use declarative constraints for local facts, stored procedures or service transactions for multi-row workflows, and triggers for automatic row-level side effects such as audit logging.

Rule class	Best mechanism	Examples
Single-row validity	NOT NULL, CHECK, DEFAULT	Rating 1-5; end_at > start_at; status in permitted values.
Uniqueness	PRIMARY KEY / UNIQUE	Email; one allocation per request; one session per booking.
Relationship integrity	FOREIGN KEY	Booking references availability, allocation, student and mentor.
Cross-row / cross-table workflow	Stored procedure + transaction + locks	Accept request; check capacity; reserve slot; prevent overlap.
Automatic row side effect	AFTER / BEFORE trigger	Audit status changes; reject malformed row values.
Access control	Application authorization + DB grants / views	Only target mentor accepts a request; public mentor directory.
Asynchronous side effect	Transactional outbox + worker	Send email/push after successful commit.
Search / recommendations	Neo4j or optional vector search, revalidated in MySQL	Find candidate mentors; never authorise or confirm booking.

9.1 Suggested logical constraints

Table	Constraint / index	Purpose
user_account	PRIMARY KEY(user_id);	Unique identity.

Table	Constraint / index	Purpose
	UNIQUE(email_normalized)	
student_profile	PRIMARY KEY(student_id); UNIQUE(user_id); FK user_id	At most one student profile per account.
mentor_profile	PRIMARY KEY(mentor_id); UNIQUE(user_id); FK user_id	At most one mentor profile per account.
student_skill	UNIQUE(student_id, skill_id); FKs to student and skill	No duplicate skill link.
mentor_skill	UNIQUE(mentor_id, skill_id); FKs to mentor and skill	No duplicate expertise link.
mentorship_request	FK student_id, mentor_id, domain_id; CHECK status	Request ownership and valid states.
mentor_allocation	UNIQUE(request_id); FKs; CHECK status	One allocation per accepted request.
mentor_availability	CHECK(end_at > start_at); FK mentor_id; index (mentor_id, start_at, end_at, status)	Valid time window and schedule access.
booking	FK slot_id, allocation_id, student_id, mentor_id; CHECK(end_at > start_at)	Booking references valid records.
booking	UNIQUE(active_slot_id) generated from CONFIRMED status	Only one active booking per slot while preserving cancelled history.
mentorship_session	UNIQUE(booking_id); FK booking_id; CHECK status	At most one session record per booking.
session_feedback	UNIQUE(session_id, submitted_by_user_id); FKs	One feedback per participant per session.
mentor_review	UNIQUE(session_id, student_id); CHECK(rating BETWEEN 1 AND 5)	One review per student per session and bounded rating.
notification_outbox	UNIQUE(event_key, recipient_user_id); index(delivery_status, created_at)	Deduplication and worker polling.

9.2 Example DDL fragments

The fragments below show the important enforcement pattern. Adjust data types and names to your final schema. MySQL enforces CHECK constraints from version 8.0.16 onward; confirm the deployed server version before relying on them.

Example: declarative constraints

```
-- A cancelled booking remains in the table, but releases its slot.
ALTER TABLE booking
  ADD COLUMN active_slot_id BIGINT
    GENERATED ALWAYS AS (
      CASE WHEN status = 'CONFIRMED' THEN slot_id ELSE NULL END
    ) STORED,
  ADD CONSTRAINT uq_booking_active_slot UNIQUE (active_slot_id),
  ADD CONSTRAINT ck_booking_time CHECK (end_at > start_at);

ALTER TABLE mentor_allocation
  ADD CONSTRAINT uq_allocation_request UNIQUE (request_id);

ALTER TABLE mentorship_session
  ADD CONSTRAINT uq_session_booking UNIQUE (booking_id);

ALTER TABLE mentor_review
  ADD CONSTRAINT uq_review_student_session UNIQUE (session_id, student_id),
  ADD CONSTRAINT ck_review_rating CHECK (rating BETWEEN 1 AND 5);

ALTER TABLE mentor_availability
  ADD CONSTRAINT ck_availability_time CHECK (end_at > start_at);

-- Use equivalent CHECKs for each controlled lifecycle vocabulary.
ALTER TABLE mentorship_request
  ADD CONSTRAINT ck_request_status
  CHECK (status IN ('PENDING','ACCEPTED','REJECTED','CANCELLED'));
```

```
ALTER TABLE mentor_allocation
  ADD CONSTRAINT ck_allocation_status
  CHECK (status IN ('ACTIVE','ENDED'));

ALTER TABLE mentorship_session
  ADD CONSTRAINT ck_session_status
  CHECK (status IN ('SCHEDULED','COMPLETED','CANCELLED','NO_SHOW'));
```

Why generated active_slot_id?

A simple UNIQUE(slot_id) would also block rebooking a slot after the previous booking was cancelled if cancelled rows are retained. A nullable generated key is populated only while status is CONFIRMED; MySQL unique indexes permit multiple NULL values, so historical cancelled rows remain while one active reservation is allowed.

9.3 What constraints cannot safely do alone

- A CHECK constraint cannot reliably compare a booking against all other bookings to detect overlapping time intervals.
- A CHECK constraint cannot verify that the selected mentor is the same mentor who owns the allocation and the availability slot.
- A UNIQUE key can enforce one active slot reservation through a generated key, but it does not by itself prevent a student booking two different overlapping slots.
- Capacity counts, state transitions and notifications span several rows; they require a transaction and consistent locking policy.
- A trigger is not a substitute for a complete business transaction or an authorization layer.

10. Trigger Design and MySQL Trigger Examples

A trigger is a database program that runs automatically as a side effect of INSERT, UPDATE or DELETE. In MySQL it is associated with a table, an event and a timing (BEFORE or AFTER), and executes FOR EACH ROW. This matches the DBMS Unit 2 trigger model.

10.1 Where triggers fit in AlumniConnect

Trigger use	Good fit?	Reason
Write audit record after request/booking status changes	Yes	Automatic row-level effect; cannot be forgotten by a caller performing an allowed update.
Reject invalid scalar value before insert	Yes	A row-local check such as rating range or invalid time ordering. Prefer CHECK where possible.
Create allocation, update request, send notification and reserve slot	No	This is a multi-step workflow; use a stored procedure/transaction so the full operation is explicit.
Send email or call Redis/Neo4j	No	A database trigger should not perform network side effects. Use an outbox worker after commit.
Perform a complex overlap search without coordinating locks	No	A trigger alone does not give a safe cross-request concurrency protocol.

10.2 Trigger T1: audit request status transitions

This AFTER UPDATE trigger creates an audit record whenever a request status changes. It records the old and new status, timestamp and database principal. The authenticated application actor should also be recorded explicitly by the business procedure or event log; CURRENT_USER() is the database account, not necessarily the human actor.

MySQL 8.x example: request-status audit trigger

```
CREATE TABLE IF NOT EXISTS audit_log (
  audit_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
  entity_type   VARCHAR(40) NOT NULL,
  entity_id     BIGINT NOT NULL,
  action_type   VARCHAR(40) NOT NULL,
  old_values    JSON NULL,
  new_values    JSON NULL,
  db_principal  VARCHAR(288) NOT NULL,
  occurred_at   TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6)
) ENGINE=InnoDB;

DELIMITER $$
CREATE TRIGGER trg_request_status_audit_au
AFTER UPDATE ON mentorship_request
FOR EACH ROW
BEGIN
  IF NOT (OLD.status <=> NEW.status) THEN
    INSERT INTO audit_log (
      entity_type, entity_id, action_type,
      old_values, new_values, db_principal, occurred_at
    ) VALUES (
      'MENTORSHIP_REQUEST', NEW.request_id, 'STATUS_CHANGED',
      JSON_OBJECT('status', OLD.status),
      JSON_OBJECT('status', NEW.status),
      CURRENT_USER(), CURRENT_TIMESTAMP(6)
    );
  END IF;
END$$
DELIMITER ;
```

Expected behaviour: updating PENDING to ACCEPTED creates one audit row; updating non-status fields does not; attempting an invalid transition is rejected by the stored procedure before the update is performed.

Record the real application actor separately

CURRENT_USER() records the MySQL connection principal, not the logged-in student, mentor or administrator. Create a business_event_log table with actor_user_id, entity_type, entity_id, action_type, payload and occurred_at. Insert into it from authorised procedures using the authenticated actor ID. This produces both a low-level row-change trail and a human-attributed business-event trail.

10.3 Trigger T2: last-line guard for mentor reviews

The main validation belongs in sp_submit_mentor_review. This optional BEFORE INSERT trigger protects the table if an unexpected code path attempts a direct insert. It verifies that the session is completed and the submitted student is the student tied to the booking.

MySQL 8.x example: review validation trigger

```
DELIMITER $$
CREATE TRIGGER trg_mentor_review_validate_bi
BEFORE INSERT ON mentor_review
FOR EACH ROW
BEGIN
  DECLARE v_session_status VARCHAR(20);
  DECLARE v_student_id BIGINT;
  DECLARE v_found BOOLEAN DEFAULT TRUE;

  DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET v_found = FALSE;

  SELECT ms.status, b.student_id
    INTO v_session_status, v_student_id
  FROM mentorship_session ms
  JOIN booking b ON b.booking_id = ms.booking_id
  WHERE ms.session_id = NEW.session_id;

  IF v_found = FALSE THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Review session or booking not found';
  ELSEIF v_session_status <> 'COMPLETED' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'A review requires a completed session';
  ELSEIF v_student_id <> NEW.student_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Only the session student may review the mentor';
  END IF;
END$$
DELIMITER ;
```

10.4 Trigger implementation cautions

- MySQL triggers execute once per affected row. A bulk UPDATE can therefore produce many audit rows.
- Trigger writes participate in the same transaction as the triggering statement. If the transaction rolls back, trigger writes roll back too.
- Keep triggers short and deterministic. Do not call external services or attempt email delivery.
- Do not place COMMIT, ROLLBACK or START TRANSACTION inside a trigger.
- Avoid hidden multi-table workflows and recursive trigger patterns. Put business orchestration in a procedure.
- Do not rely on CURRENT_USER() to identify the application's human actor; it identifies the database principal.
- Restrict direct DML on sensitive tables so application code cannot bypass procedures and authorization checks.

11. Stored Procedure Catalogue and Implementation Examples

A stored procedure is a named, stored collection of SQL statements that can accept IN, OUT or INOUT parameters and be called by the application using CALL. The Unit 2 material emphasises that procedures support modularity, reduce duplicated SQL and can enforce complex constraints. AlumniConnect should use procedures for workflows that must validate and update several related rows consistently.

11.1 Recommended procedure inventory

Procedure	Purpose and key rule
sp_submit_mentorship_request	Validate active student/mentor, reject duplicate pending request, insert PENDING request and REQUEST_SUBMITTED outbox event.
sp_accept_mentorship_request	Lock actors and request; check PENDING and mentor capacity; create one allocation; update request; queue student notification.
sp_reject_mentorship_request	Verify target mentor and PENDING state; record reason/decision time; queue student notification.
sp_cancel_mentorship_request	Allow only the requesting student to cancel a PENDING request; retain history.
sp_publish_mentor_slot	Validate mentor ownership, future time range and no overlapping open/active availability.
sp_create_booking	Validate active allocation and open slot; lock participants and slot; prevent mentor/student overlap; insert booking/session/outbox; reserve slot atomically.
sp_cancel_booking	Authorize participant/administrator, enforce cancellation cutoff, mark booking/session cancelled and release future slot if policy permits.
sp_reschedule_booking	Preserve old booking; cancel or mark superseded and create replacement booking in a controlled workflow.
sp_record_session_outcome	Allow assigned mentor to set COMPLETED/CANCELLED/NO_SHOW and add structured observations.
sp_submit_session_feedback	Allow either actual participant to submit feedback only after the session has taken place.
sp_submit_mentor_review	Allow the actual student to add one review for a completed session.
sp_end_allocation	End an active allocation with actor, reason and timestamp; prevent new booking.

11.2 Procedure parameter design

Parameter mode	Meaning	AlumniConnect example
IN	Caller supplies a value; procedure reads it.	IN p_request_id BIGINT; IN p_actor_user_id BIGINT.
OUT	Procedure returns a value to caller.	OUT p_booking_id BIGINT; useful if API expects a generated ID.
INOUT	Caller supplies a value that can be changed and returned.	Possible for counters or configurable pagination, but less common for workflow procedures.

11.3 Procedure P1: accept a mentorship request

This example assumes student_profile(student_id, user_id), mentor_profile(mentor_id, user_id, max_active_mentees), mentorship_request(request_id, student_id, mentor_id, status, decision_at), mentor_allocation(allocation_id, request_id, student_id, mentor_id, status, created_at), and notification_outbox(event_type, aggregate_type, aggregate_id, recipient_user_id, payload, delivery_status, created_at).

Transaction ownership

This procedure owns its transaction. Call it as a standalone operation rather than from inside another open transaction. Every other procedure that creates or ends allocations must lock the mentor profile row before changing the active-allocation count.

MySQL 8.x example: atomic request acceptance

```
DELIMITER $$
CREATE PROCEDURE sp_accept_mentorship_request(
  IN p_request_id BIGINT,
  IN p_actor_user_id BIGINT
)
proc: BEGIN
  DECLARE v_student_id BIGINT;
  DECLARE v_mentor_id BIGINT;
  DECLARE v_student_user_id BIGINT;
  DECLARE v_mentor_user_id BIGINT;
  DECLARE v_max_active INT;
  DECLARE v_active_count INT;
  DECLARE v_status VARCHAR(20);
  DECLARE v_exists INT DEFAULT 0;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SELECT COUNT(*) INTO v_exists
  FROM mentorship_request
  WHERE request_id = p_request_id;

  IF v_exists = 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Mentorship request not found';
  END IF;

  -- Initial read obtains the participant IDs; values are re-read under lock.
  SELECT student_id, mentor_id
    INTO v_student_id, v_mentor_id
  FROM mentorship_request
  WHERE request_id = p_request_id;

  START TRANSACTION;

  -- Fixed lock order: student, then mentor, then request.
  SELECT user_id INTO v_student_user_id
  FROM student_profile
  WHERE student_id = v_student_id
  FOR UPDATE;

  SELECT user_id, max_active_mentees
    INTO v_mentor_user_id, v_max_active
```

```
  FROM mentor_profile
  WHERE mentor_id = v_mentor_id
  FOR UPDATE;

  IF v_mentor_user_id <> p_actor_user_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Only the addressed mentor may accept this request';
  END IF;

  SELECT status INTO v_status
  FROM mentorship_request
  WHERE request_id = p_request_id
  FOR UPDATE;

  IF v_status <> 'PENDING' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Request is not pending';
  END IF;

  SELECT COUNT(*) INTO v_active_count
  FROM mentor_allocation
  WHERE mentor_id = v_mentor_id
    AND status = 'ACTIVE';

  IF v_active_count >= v_max_active THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Mentor capacity has been reached';
  END IF;

  INSERT INTO mentor_allocation
    (request_id, student_id, mentor_id, status, created_at)
  VALUES
    (p_request_id, v_student_id, v_mentor_id,
     'ACTIVE', CURRENT_TIMESTAMP(6));

  UPDATE mentorship_request
  SET status = 'ACCEPTED', decision_at = CURRENT_TIMESTAMP(6)
  WHERE request_id = p_request_id;

  -- Human actor is captured explicitly; CURRENT_USER() in a trigger is the DB login.
  INSERT INTO business_event_log
    (actor_user_id, entity_type, entity_id, action_type,
     payload, occurred_at)
  VALUES
    (p_actor_user_id, 'MENTORSHIP_REQUEST', p_request_id,
     'REQUEST_ACCEPTED', JSON_OBJECT('student_id', v_student_id,
                                     'mentor_id', v_mentor_id),
     CURRENT_TIMESTAMP(6));

  INSERT INTO notification_outbox
    (event_type, aggregate_type, aggregate_id,
     recipient_user_id, payload, delivery_status, created_at)
  VALUES
    ('REQUEST_ACCEPTED', 'MENTORSHIP_REQUEST', p_request_id,
     v_student_user_id,
     JSON_OBJECT('request_id', p_request_id),
     'PENDING', CURRENT_TIMESTAMP(6));

  COMMIT;
END$$
DELIMITER ;
```

The unique constraint on mentor_allocation.request_id is still required. The procedure's row locks protect the capacity check only when all allocation-creation and allocation-ending paths follow the same mentor-row locking convention.

Complete the account guard in the final schema

Before acceptance, also verify that both linked user accounts are ACTIVE and the mentor profile is verified/eligible. Those account-status columns were not present in the compact assumptions for this code sample, so add the corresponding joins and checks from your final user_account schema.

11.4 Procedure P2: create a booking safely

The following is a compact implementation template. It assumes the booking caller is the student, all bookings are tied to an ACTIVE allocation, an availability slot is one-to-one, and status CONFIRMED represents an active reservation. The same lock order must be used by all booking/cancellation/reschedule routines.

Production checks still required

Also validate that the student and mentor user accounts are ACTIVE, the mentor is eligible to accept sessions, the allocation remains valid, and the idempotency key has not already been consumed. These checks must run within the same transaction or be protected by a suitable lock/unique key.

MySQL 8.x example: atomic booking with participant and slot locks

```
DELIMITER $$
CREATE PROCEDURE sp_create_booking(
  IN p_allocation_id BIGINT,
  IN p_slot_id BIGINT,
  IN p_actor_user_id BIGINT
)
proc: BEGIN
  DECLARE v_student_id BIGINT;
  DECLARE v_mentor_id BIGINT;
  DECLARE v_student_user_id BIGINT;
  DECLARE v_mentor_user_id BIGINT;
  DECLARE v_allocation_status VARCHAR(20);
  DECLARE v_slot_status VARCHAR(20);
  DECLARE v_start DATETIME(6);
  DECLARE v_end DATETIME(6);
  DECLARE v_conflicts INT DEFAULT 0;
  DECLARE v_match INT DEFAULT 0;
  DECLARE v_booking_id BIGINT;
  DECLARE v_slot_mentor_id BIGINT;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SELECT COUNT(*) INTO v_match
  FROM mentor_allocation a
  JOIN mentor_availability s ON s.mentor_id = a.mentor_id
  WHERE a.allocation_id = p_allocation_id
    AND s.slot_id = p_slot_id;

  IF v_match = 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Slot does not belong to allocation mentor';
  END IF;

  SELECT a.student_id, a.mentor_id
    INTO v_student_id, v_mentor_id
  FROM mentor_allocation a
  WHERE a.allocation_id = p_allocation_id;

  START TRANSACTION;

  -- Fixed lock order serializes bookings for the same participants.
  SELECT user_id INTO v_student_user_id
  FROM student_profile
  WHERE student_id = v_student_id
  FOR UPDATE;

  SELECT user_id INTO v_mentor_user_id
  FROM mentor_profile
  WHERE mentor_id = v_mentor_id
  FOR UPDATE;

  IF v_student_user_id <> p_actor_user_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Only the allocation student may create this booking';
  END IF;

  SELECT status INTO v_allocation_status
  FROM mentor_allocation
  WHERE allocation_id = p_allocation_id
  FOR UPDATE;

  IF v_allocation_status <> 'ACTIVE' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Allocation is not active';
  END IF;

  SELECT status, start_at, end_at, mentor_id
    INTO v_slot_status, v_start, v_end, v_slot_mentor_id
  FROM mentor_availability
  WHERE slot_id = p_slot_id
  FOR UPDATE;

  IF v_slot_status <> 'OPEN' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Availability slot is no longer open';
  END IF;

  IF v_slot_mentor_id <> v_mentor_id OR v_start <= CURRENT_TIMESTAMP(6) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Slot owner or start time is invalid';
  END IF;
```

```
  SELECT COUNT(*) INTO v_conflicts
  FROM booking b
  WHERE b.status = 'CONFIRMED'
    AND b.start_at < v_end
    AND b.end_at > v_start
    AND (b.student_id = v_student_id OR b.mentor_id = v_mentor_id);

  IF v_conflicts > 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Student or mentor has an overlapping booking';
  END IF;

  INSERT INTO booking
    (slot_id, allocation_id, student_id, mentor_id,
     start_at, end_at, status, created_at)
  VALUES
    (p_slot_id, p_allocation_id, v_student_id, v_mentor_id,
     v_start, v_end, 'CONFIRMED', CURRENT_TIMESTAMP(6));

  SET v_booking_id = LAST_INSERT_ID();

  UPDATE mentor_availability
  SET status = 'BOOKED'
  WHERE slot_id = p_slot_id;

  INSERT INTO mentorship_session (booking_id, status)
  VALUES (v_booking_id, 'SCHEDULED');

  INSERT INTO business_event_log
    (actor_user_id, entity_type, entity_id, action_type,
     payload, occurred_at)
  VALUES
    (p_actor_user_id, 'BOOKING', v_booking_id, 'BOOKING_CONFIRMED',
     JSON_OBJECT('allocation_id', p_allocation_id, 'slot_id', p_slot_id),
     CURRENT_TIMESTAMP(6));

  INSERT INTO notification_outbox
    (event_type, aggregate_type, aggregate_id,
     recipient_user_id, payload, delivery_status, created_at)
  VALUES
    ('BOOKING_CONFIRMED', 'BOOKING', v_booking_id,
     v_student_user_id,
     JSON_OBJECT('booking_id', v_booking_id),
     'PENDING', CURRENT_TIMESTAMP(6)),
    ('BOOKING_CONFIRMED', 'BOOKING', v_booking_id,
     v_mentor_user_id,
     JSON_OBJECT('booking_id', v_booking_id),
     'PENDING', CURRENT_TIMESTAMP(6));

  COMMIT;
  SELECT v_booking_id AS booking_id;
END$$
DELIMITER ;
```

Important implementation note

The example demonstrates the locking pattern, not a complete drop-in schema. Add an idempotency key and a unique generated active_slot_id constraint. Every booking writer must lock the student profile and mentor profile in the same order before checking overlaps; otherwise two transactions can still race on different slots.

11.5 Procedure P3: submit a mentor review

MySQL 8.x example: review eligibility and participant validation

```
DELIMITER $$
CREATE PROCEDURE sp_submit_mentor_review(
  IN p_session_id BIGINT,
  IN p_actor_user_id BIGINT,
  IN p_rating TINYINT,
  IN p_review_text VARCHAR(2000)
)
proc: BEGIN
  DECLARE v_student_id BIGINT;
  DECLARE v_student_user_id BIGINT;
  DECLARE v_session_status VARCHAR(20);
  DECLARE v_count INT DEFAULT 0;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  IF p_rating < 1 OR p_rating > 5 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Rating must be between 1 and 5';
  END IF;
```

```
  SELECT COUNT(*) INTO v_count
  FROM mentorship_session ms
  JOIN booking b ON b.booking_id = ms.booking_id
  JOIN student_profile sp ON sp.student_id = b.student_id
  WHERE ms.session_id = p_session_id;

  IF v_count = 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Session not found';
  END IF;

  START TRANSACTION;

  SELECT b.student_id, sp.user_id, ms.status
    INTO v_student_id, v_student_user_id, v_session_status
  FROM mentorship_session ms
  JOIN booking b ON b.booking_id = ms.booking_id
  JOIN student_profile sp ON sp.student_id = b.student_id
  WHERE ms.session_id = p_session_id
  FOR UPDATE;

  IF v_student_user_id <> p_actor_user_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Only the session student may review the mentor';
  END IF;

  IF v_session_status <> 'COMPLETED' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'A mentor review requires a completed session';
  END IF;

  INSERT INTO mentor_review
    (session_id, student_id, rating, review_text, created_at)
  VALUES
    (p_session_id, v_student_id, p_rating,
     p_review_text, CURRENT_TIMESTAMP(6));

  INSERT INTO business_event_log
    (actor_user_id, entity_type, entity_id, action_type,
     payload, occurred_at)
  VALUES
    (p_actor_user_id, 'MENTOR_REVIEW', p_session_id, 'REVIEW_SUBMITTED',
     JSON_OBJECT('rating', p_rating), CURRENT_TIMESTAMP(6));

  COMMIT;
END$$
DELIMITER ;
```

Keep UNIQUE(session_id, student_id) on mentor_review even when the procedure checks for eligibility. The database uniqueness rule is the final protection against two concurrent duplicate submissions.

11.6 Cancellation and rescheduling procedure contract

Operation	Required steps inside one transaction
Cancel booking	Lock student, then mentor, then booking and slot; authorize actor; check current state and cancellation cutoff; set booking/session to CANCELLED; release slot only if future and policy permits; write outbox events for both participants; commit.
Reschedule booking	Validate original booking; preserve it as CANCELLED/RESCHEDULED; validate new slot with the same booking checks; create new booking and session; link replacement to original; send notification; commit.
Record session outcome	Lock session/booking; verify mentor is participant; require SCHEDULED; set COMPLETED/CANCELLED/NO_SHOW; store outcome notes and timestamps; queue relevant notification; commit.
End allocation	Lock student and mentor in the agreed order; verify ACTIVE; set ENDED with actor/time/reason; block future bookings; preserve existing session history.

12. Transactions, ACID, Isolation and Concurrency

The DBMS Unit 4 material presents transactions as all-or-none units and introduces ACID, concurrency problems, locking, two-phase locking and deadlocks. AlumniConnect should demonstrate these concepts in request acceptance and especially in booking.

12.1 ACID applied to AlumniConnect

Property	Meaning in this platform	Concrete example
Atomicity	All required writes commit together or none do.	Booking insert, slot state change, session insert and outbox events succeed or all roll back.
Consistency	A committed transaction preserves declared constraints and business invariants.	A booking cannot point to an inactive allocation or a slot owned by another mentor.
Isolation	Concurrent operations cannot observe or create invalid intermediate states.	Two students racing for one slot cannot both confirm it.
Durability	Once commit succeeds, changes survive a server restart/failure under the database's durability configuration.	A confirmed booking remains present after a crash/restart.

12.2 Concurrency anomalies relevant to the platform

Anomaly	AlumniConnect example	Control
Lost update	Two administrators or procedures update the same slot/booking state, with one overwriting the other.	Row locks; state re-check; controlled procedures.
Dirty read	A student sees a booking that another transaction later rolls back.	Do not use READ UNCOMMITTED for business reads; use committed reads.
Non-repeatable read	A screen reads allocation status twice and sees ACTIVE then ENDED.	Use a transaction snapshot or locking read where a stable decision is required.
Phantom / race on overlap	Two transactions each see no existing overlap, then both insert different overlapping slots.	Lock participant parent rows in a fixed order; all writers follow the same rule; add appropriate indexes.
Duplicate submission	Browser retries booking after timeout and inserts a second booking.	Idempotency key + unique key + return existing operation result.

12.3 Locking strategy

- Use InnoDB row-level locks for the records that represent the resources being changed.
- Use SELECT ... FOR UPDATE on the mentor availability row to serialize attempts to claim the same slot.
- Also lock the student profile and mentor profile rows before overlap checks. These parent rows act as serialization points for all booking operations involving either participant.
- Adopt one fixed lock order for every related procedure, for example: student profile -> mentor profile -> allocation -> availability slot -> booking/session.
- Re-read status after acquiring locks. A value read before the lock is only a discovery hint, not proof that it is still valid.
- Keep the transaction short. Do not call email providers, Redis, Neo4j or remote APIs while holding database locks.

12.4 Isolation level guidance

InnoDB commonly uses REPEATABLE READ as its default isolation level, but confirm the actual server setting. For this application, use explicit locking reads for booking/acceptance decisions and never rely on a plain SELECT

alone to reserve a resource. READ COMMITTED can be appropriate for some application reads; SERIALIZABLE should not be enabled globally just to solve one booking race. Test the actual isolation configuration with two independent database sessions.

12.5 Deadlocks and retry behaviour

Two-phase locking can ensure conflict-serializable behaviour but does not eliminate deadlocks. The Unit 4 slides demonstrate a deadlock when two transactions obtain locks in different orders and MySQL rolls one transaction back. AlumniConnect should therefore use a consistent lock order, short transactions, suitable indexes and bounded retry for transient deadlock errors.

MySQL condition	Recommended application behaviour
1213 - Deadlock found	Retry the entire transaction/procedure with a small bounded retry count and jitter. Do not retry only the final statement.
1205 - Lock wait timeout	Roll back the full business operation, return a retryable error where appropriate, and log contention.
1062 - Duplicate key	Interpret by constraint name: slot already reserved, request already allocated, review already submitted, or idempotency replay. Return a meaningful result.
45000 - Business rule SIGNAL	Do not blindly retry. Return a domain validation error to the UI.

12.6 Transactional outbox pattern

7. Within the same SQL transaction as the state change, insert an outbox row with event type, aggregate ID, recipient and payload.
8. Commit the business record and outbox row together.
9. A worker polls pending outbox rows after commit, sends the notification, and marks delivery status.
10. Use event IDs and recipient-level unique keys so worker retries do not duplicate deliveries.
11. If delivery repeatedly fails, retain the event for retry/dead-letter inspection; never undo the accepted request or confirmed booking.

13. Views, Functions, Indexes and Advanced SQL

13.1 Views

View	Purpose	Example use
v_public_mentor_directory	Expose only approved public profile fields and searchable skills/domains.	Student mentor discovery.
v_student_mentorship_history	Combine requests, allocations, bookings and sessions for one student.	Student dashboard.
v_mentor_booking_calendar	Show active confirmed bookings for a mentor.	Mentor calendar.
v_mentor_rating_summary	Aggregate eligible completed-session reviews.	Mentor card and admin analytics.
v_admin_request_funnel	Count request states and conversions into allocations/sessions.	Platform analytics.

A view can present a stable query interface and hide columns that should not be exposed. A view is not automatically a security boundary: grant access to the view, restrict base-table privileges and verify that the view definition matches the privacy policy.

13.2 Stored functions

Use a stored function for a reusable scalar calculation, not for a multi-step workflow. Candidate examples are profile completeness, a normalised rating metric or a small derived label. Functions should not be used to send notifications or modify booking state.

Illustrative read-only function for mentor rating summary

```
DELIMITER $$
CREATE FUNCTION fn_mentor_average_rating(p_mentor_id BIGINT)
RETURNS DECIMAL(3,2)
READS SQL DATA
BEGIN
 DECLARE v_avg DECIMAL(3,2);

 SELECT ROUND(AVG(mr.rating), 2)
  INTO v_avg
 FROM mentor_review mr
 JOIN mentorship_session ms ON ms.session_id = mr.session_id
 JOIN booking b ON b.booking_id = ms.booking_id
 WHERE b.mentor_id = p_mentor_id
  AND ms.status = 'COMPLETED';

 RETURN COALESCE(v_avg, 0.00);
END$$
DELIMITER ;
```

13.3 Advanced SQL showcases

Concept	Business question to demonstrate
JOINs	Show mentor profiles with company, department, skills and domains.
GROUP BY / HAVING	Find mentors with a minimum number of completed sessions and an average rating above a threshold.
Nested / correlated subquery	Find students who have not received any allocation; find mentors with experience above the average in their domain.
CTE	Analyse Requests Submitted -> Accepted -> Allocations Created -> Sessions Completed.
Window functions	Rank mentors within each mentorship domain by rating, session count or matching score.
Full-text search	Search alumni biographies, technical experience and mentoring descriptions.
EXPLAIN / indexes	Compare mentor discovery, booking-calendar and mentorship-history query plans.

13.4 Indexing guidelines

- Index foreign keys and frequently filtered status/time columns: (mentor_id, status, start_at), (student_id, status, start_at), and (allocation_id, status).
- Use composite indexes in the order that matches actual predicates and sort order; verify with EXPLAIN rather than indexing every column.
- Make the overlap predicate sargable where possible and inspect actual plans on realistic data volumes.
- Keep unique constraints even when application checks exist; they protect correctness under concurrency.
- Measure write overhead and storage cost before adding redundant indexes.

14. Redis, Neo4j and Cross-Database Consistency

The blueprint assigns clear responsibilities to each data store. The business rules must preserve that boundary.

Store	What it may do	What it must not do
MySQL / InnoDB	Authoritative identities, profiles, requests, allocations, availability, bookings, sessions,	Allow invalid workflow transitions or rely only on frontend validation.

Store	What it may do	What it must not do
	feedback, reviews, outbox and audit.	
Redis	Cache mentor cards, domain/skill data and recommendation results; store short-lived login challenges, rate-limit counters and idempotency records.	Be the final authority for slot availability, booking confirmation, allocation status or user authorization.
Neo4j	Represent Student, Mentor, Skill, Domain, Company and Department nodes/relationships to traverse relevant expertise paths and explain recommendations.	Create authoritative requests, allocations or bookings.
Optional vector search	Retrieve semantically similar alumni descriptions from a natural-language student goal.	Bypass SQL eligibility or privacy checks.

14.1 Cross-store rules

- MySQL wins if data differs between stores.
- Redis entries are rebuildable and expire through TTL. Invalidate caches after the authoritative SQL commit.
- Neo4j is a projection of selected MySQL entities. Changes should be propagated asynchronously and may be eventually consistent.
- A recommendation from Neo4j or vector search is a candidate only. Before display or action, the backend must re-check mentor account status, privacy/visibility, current skills where required and current capacity in MySQL.
- Never use a Redis cache hit as proof that a slot is still free. Booking procedure must lock and validate MySQL rows.

15. Security, Privacy and Data Lifecycle

15.1 Authorization rules

Action	Student	Mentor	Administrator
Edit own profile	Own student profile	Own mentor profile	Moderate/assist through audited admin workflow
Discover public mentors	Yes	Preview public directory	Yes
Submit mentorship request	Yes	No	No (unless a distinct administrative workflow is approved)
Accept/reject request	No	Only requests addressed to self	Override only with explicit reason and audit
Publish availability	No	Own availability	May block/correct through audited workflow
Book slot	Own active allocation	No, except a separately approved assisted workflow	No direct bypass
Record session outcome	No	Assigned mentor	Audited correction only
Submit session feedback	Only if participant	Only if participant	No
Submit mentor review	Only actual student participant	No	No; moderation is separate from authorship
Manage taxonomy/accounts	No	No	Yes, permission-scoped

15.2 Privacy and retention

- Use least privilege: students see public mentor profile attributes and their own private records; mentors see relevant student information for their accepted allocations and sessions.
- Do not expose private email, phone, internal notes or admin-only audit fields in the public mentor directory.
- Use soft deactivation for accounts and retain required business history. Define a documented retention/deletion policy for personal data separately.
- Review text and session notes should be access-controlled and should not contain passwords, tokens, financial account information or unrelated sensitive information.
- Audit access to administrative views and sensitive exports where practical.

15.3 Data deletion policy

Do not cascade-delete requests, allocations, bookings, sessions, feedback or reviews when an account is deactivated. Prefer status changes and restricted deletion. If legal/privacy deletion is later required, define a controlled anonymisation procedure that preserves referential integrity and legitimate aggregate reporting while removing or minimising personal identifiers.

16. Verification and Acceptance Test Catalogue

The business rules should be converted into repeatable SQL integration tests and API tests. Tests that involve concurrency must use at least two separate database connections, not sequential calls in one session.

Test ID	Scenario	Expected result
BR-T01	Student submits a request to an active mentor.	One PENDING request; mentor outbox notification exists.
BR-T02	Student submits the same pending student-mentor-domain request again.	Duplicate blocked or deterministic existing-request response.
BR-T03	A student attempts to accept their own request.	Rejected; no allocation or notification created.
BR-T04	Target mentor accepts a PENDING request.	One ACTIVE allocation; request ACCEPTED; one outbox notification; all commit together.
BR-T05	Two acceptance attempts race on the same request.	Exactly one allocation; second returns already-processed/duplicate result.
BR-T06	Mentor capacity is already full.	Acceptance rejected; request remains PENDING.
BR-T07	A rejected/cancelled request is queried for history.	Historical row remains available.
BR-T08	Student tries to book a slot belonging to another mentor.	Rejected; no booking inserted.
BR-T09	Student tries to book while allocation is ENDED.	Rejected.
BR-T10	Two students book the same slot simultaneously.	Exactly one CONFIRMED booking; one fails cleanly.
BR-T11	Student books two overlapping slots with different mentors concurrently.	At most one confirmed booking; student-row lock serializes the check.
BR-T12	Mentor has an existing overlapping booking.	New booking rejected.
BR-T13	A 10:00-10:30 slot is followed by 10:30-11:00.	Both can be confirmed; adjacent half-open intervals do not overlap.
BR-T14	Client retries booking with same idempotency key.	Original booking ID/result returned; no second booking/session.
BR-T15	Failure occurs after booking insert but before slot update.	Full transaction rolls back; no partial booking/session/outbox.
BR-T16	Booking is cancelled within allowed cutoff.	Booking retained as CANCELLED; slot reopens if policy permits; both participants notified.

Test ID	Scenario	Expected result
BR-T17	Booking cancellation occurs after cutoff.	Rejected or escalated according to configured policy.
BR-T18	A confirmed booking produces a session.	Exactly one SCHEDULED session.
BR-T19	Mentor records completion and required notes.	Session becomes COMPLETED and outcome is retained.
BR-T20	Student attempts to mark another mentor's session completed.	Rejected.
BR-T21	Feedback submitted before session completion.	Rejected.
BR-T22	Non-participant submits session feedback.	Rejected.
BR-T23	Same participant submits feedback twice.	Second insert rejected by unique constraint or revision policy.
BR-T24	Review submitted for CANCELLED or NO_SHOW session.	Rejected.
BR-T25	Student reviews a completed session they attended.	One review accepted.
BR-T26	Student submits a second review for the same session.	Rejected by unique(session_id, student_id).
BR-T27	Rating is 0 or 6.	Rejected by CHECK/procedure.
BR-T28	Outbox worker is offline during booking.	Booking still commits; outbox remains PENDING for retry.
BR-T29	Outbox worker retries an already-delivered event.	Duplicate delivery suppressed by idempotency key.
BR-T30	Mentor profile/skills are changed.	Relevant Redis cache invalidated/expired; recommendation projection refreshed asynchronously.
BR-T31	Neo4j recommends an inactive mentor.	Backend filters candidate using MySQL; inactive mentor not offered.
BR-T32	Admin suspends a mentor with existing sessions.	New requests/slots/actions blocked; historical data retained.
BR-T33	Audit trigger records a request status change.	One audit row has old/new status; rollback also rolls back the audit row.
BR-T34	Deadlock error 1213 occurs under concurrent update.	Whole transaction rolled back and bounded retry policy invoked.

16.1 Concurrency test procedure: same slot

12. Create one OPEN mentor slot and two students with valid ACTIVE allocations to that mentor.
13. Open two independent MySQL sessions. Start both booking calls with a synchronized barrier so they compete for the same slot.
14. Confirm that one transaction obtains the availability row lock and commits; the other waits and then observes that the slot is no longer OPEN or hits the unique active-slot constraint.
15. Verify the final data: one CONFIRMED booking, one SCHEDULED session, slot status BOOKED, and notifications only for the successful booking.

16.2 Evidence to capture for project evaluation

- DDL showing primary keys, foreign keys, CHECK and UNIQUE constraints.
- SHOW TRIGGERS and trigger execution evidence for request status audit.
- SHOW PROCEDURE STATUS and CALL examples for acceptance, booking and review.
- Two-session concurrency output proving only one booking is confirmed.
- Transaction rollback evidence showing that no partial booking or outbox event remains.

- EXPLAIN output for mentor discovery and booking-calendar queries.
- Redis cache hit/miss/TTL evidence and Neo4j candidate recommendation output with MySQL revalidation.

17. Policy Decisions to Finalise

The blueprint establishes the workflow structure, but the following operating-policy values are not fixed in the source document. The team should decide them explicitly before implementation. Until then, store them as configuration rather than burying them in SQL code.

Decision	Options to decide	Recommended implementation approach
Mentor capacity	Maximum concurrent active allocations per mentor; may vary by mentor.	max_active_mentees on mentor profile or policy table; checked under mentor-row lock.
Student active allocations	One active mentor overall, one per domain, or multiple.	Configurable limit checked under student-row lock.
Duplicate request definition	Same student + mentor only, or same student + mentor + domain.	Define a precise unique-active-request policy; preserve resolved history.
Booking horizon	How far ahead can slots be published/booked?	Configuration values in days/hours.
Cancellation cutoff	How close to start can a student/mentor cancel without admin approval?	Cancellation policy table; evaluate in procedure using stored time zone/UTC.
Slot duration and buffer	Fixed durations or mentor-defined durations; break between sessions.	Validate duration and non-overlap when publishing availability.
No-show authority	Who can mark no-show and how long after start?	Mentor action plus admin correction path; store actor and reason.
Review edit policy	Immutable after submit, editable for a short time, or revision-only.	Prefer append-only revisions with moderation audit.
Feedback visibility	Visible to both participants, mentor only, admin only or anonymised.	Define per field and enforce through API/view permissions.
Mentor verification	Self-declared profile, admin-approved profile or institution-verified alumni.	Explicit verification status and discovery eligibility rule.
Admin override scope	Which records may be corrected and under what justification?	Dedicated procedures and mandatory reason/audit metadata.
Data retention	How long to retain inactive profiles, audit and feedback?	Document retention schedule and legal/privacy review.

18. Implementation Checklist and Source Basis

18.1 Recommended implementation order

16. Freeze the logical schema and status vocabulary. Confirm the policy decisions in Section 17.
17. Create MySQL tables, primary/foreign keys, CHECK constraints, uniqueness rules and supporting indexes.
18. Implement role-based service authorization and restrict direct DML on workflow tables.
19. Implement request submission, acceptance, rejection and cancellation procedures.
20. Implement availability publishing and the atomic booking/cancellation/reschedule procedures.
21. Implement session outcome, feedback and review procedures, then add audit triggers.
22. Implement the transactional outbox worker and idempotent notification delivery.
23. Add Redis caching and Neo4j projections only after MySQL workflows are correct.
24. Run unit, integration, failure-injection and multi-session concurrency tests.
25. Capture SQL, trigger, procedure, transaction and test evidence for the DBMS project report.

18.2 Mapping to DBMS course concepts

DBMS concept	How AlumniConnect demonstrates it
Unit 1: ER model, relational schema, keys and constraints	User/role profiles, M:N skill junctions, request/allocation/booking/session relationships, PK/FK/UNIQUE/CHECK constraints.
Unit 2: Advanced SQL, views, triggers, procedures, functions	Public mentor views, request/session audit triggers, acceptance and booking procedures, rating/profile functions, analytics queries.
Unit 3: Normalisation and query optimisation	Separate identity/profile, booking/session and feedback/review concepts; functional dependencies; indexes and EXPLAIN-based plan comparisons.
Unit 4: Transactions, ACID, concurrency and locking	Atomic acceptance, atomic booking, row-level locking, overlap prevention, idempotency, deadlock retry, outbox consistency.
Unit 4: Redis and Neo4j	Redis cache and TTL; Neo4j relationship-based mentor recommendations with authoritative MySQL validation.

18.3 Source basis

This catalogue is grounded in the supplied Alumni-Student Mentorship Platform: DBMS Project Blueprint and the supplied PES DBMS course materials. The references below identify where the core source requirements and relevant academic concepts appear. Additional rule details labelled [R] are implementation recommendations developed to make the blueprint testable and enforceable.

Source	Relevant material used
Alumni-Student Mentorship Platform: DBMS Project Blueprint, pp. 2-5	Project vision; Student, Alumni/Mentor and Administrator roles; modules; modular monolith architecture; MySQL, Redis and Neo4j responsibilities.
Blueprint, pp. 8-9	Source-of-truth principle: MySQL authoritative; Redis caches; Neo4j graph projection.
Blueprint, pp. 22-25	Request lifecycle, acceptance and allocation transaction, booking validation and concurrency, offline session recording, feedback/review distinction, notification outbox.
Blueprint, pp. 27-30	DBMS Unit 1-4 mapping; triggers for audit; procedures for booking/acceptance; functions/views/CTEs/window functions; normalisation and query optimisation showcase.
PES DBMS Unit 2: Advanced SQL, pp. 213-220	Trigger definition, BEFORE/AFTER INSERT/UPDATE/DELETE, FOR EACH ROW, use cases and SIGNAL-based validation examples.
PES DBMS Unit 2: Advanced SQL, pp. 242-248 onward	Stored procedures, procedure parameters (IN/OUT/INOUT), modularity and CALL usage.
PES DBMS Unit 4: Database Transactions and ACID Properties, pp. 26-27	ACID properties and transaction all-or-none behaviour.
PES DBMS Unit 4: Lock-Based Protocols, pp. 87-102 and 134-143	Lock compatibility, deadlock behaviour, two-phase locking and row-level locking implications.

Final implementation principle

A UI check improves user experience, but it is not an integrity guarantee. The durable guarantee belongs in MySQL: constraints for local facts, procedures and transactions for cross-row workflows, locks for concurrency, triggers for appropriate automatic side effects, and an outbox for reliable external notifications.